



Proxy e Decorator

Padrões de Projeto - Aula #03

Prof. Dr. Carlos Melo

 carlos.melo@upe.br

Tabela 1.1 O espaço dos padrões de projeto

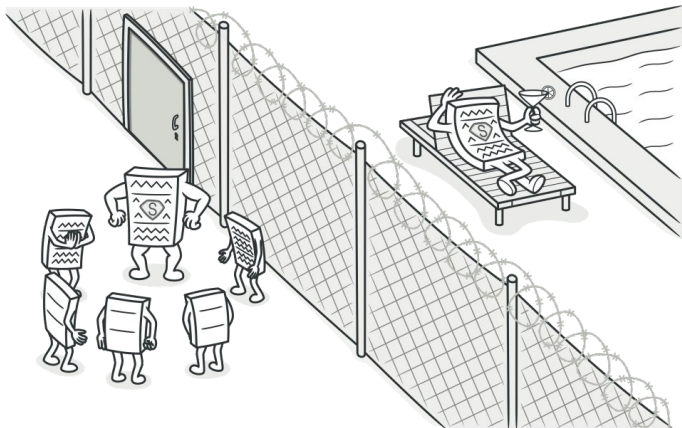
		Propósito		
		De criação	Estrutural	Comportamental
Escopo	Classe	Factory Method (112)	Adapter (class) (140)	Interpreter (231) Template Method (301)
	Objeto	Abstract Factory (95) Builder (104) Prototype (121) Singleton (130)	Adapter (object) (140) Bridge (151) Composite (160) Decorator (170) Façade (179) Flyweight (187) Proxy (198)	Chain of Responsibility (212) Command (222) Iterator (244) Mediator (257) Memento (266) Observer (274) State (284) Strategy (292) Visitor (305)



Padrões Estruturais (parte 2)

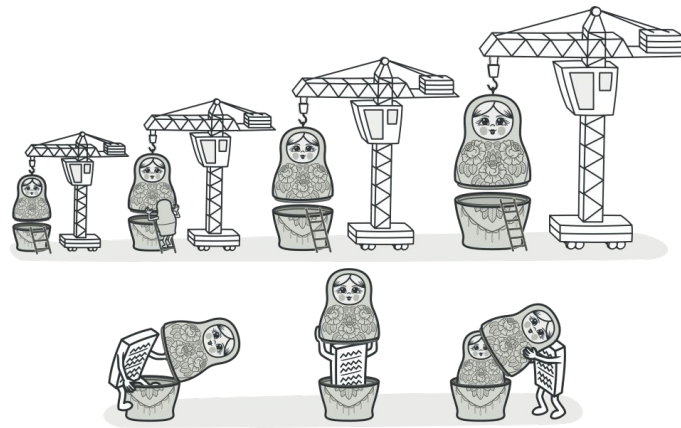
Proxy

Alguém no meio decide se e como você chega ao objeto real



Decorator

Camadas que envolvem o objeto, cada uma somando um comportamento



- Padrões estruturais tratam de como compor objetos e classes em estruturas maiores mantendo flexibilidade



Objetivos de aprendizagem

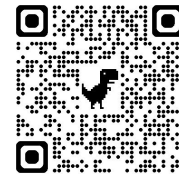
- Reconhecer quando o acesso a um objeto é o problema (Proxy) e quando a funcionalidade é o problema (Decorator)
- Implementar um proxy com carregamento tardio (*lazy loading*)
- Empilhar decoradores em tempo de execução sem tocar na classe original



Proxy - O Problema

- Sistema de armazenamento de fotos de um telescópio espacial produz imagens de dezenas de MB a alguns GB
- Uma única sessão de observação gera centenas de imagens
- Precisamos de uma aplicação web para navegar nesse acervo
- No primeiro acesso, o navegador fechou por falta de memória
- Por quê?
 - A aplicação carregou todas as imagens de uma vez





Onde está o desperdício?

- De 300 imagens carregadas, quantas o usuário realmente vai abrir?
 - 3 ou 4?
- Estamos pagando I/O de disco, banda e RAM pelo restante que ele não olhou.
- E pagamos esse custo no momento da abertura da página (o mais crítico).

Pesquisar imagens do Hubble

Tipo

- ☐ Sem-telas do Hubble
- ☐ Divulgação de imagens do Hubble
- ☐ Imagens do Hubble

Todas as categorias

Q Pesquisar por palavra-chu

- ☐ Astronomia
- ☐ Bruce McCandless
- ☐ Terra

Problema de Manutenção

Intervalo de datas

Começar:

Fim:

[Limpar filtros](#)

Pesquisar conteúdo mais recente

ORGANIZAR: DATA DE PUBLICAÇÃO **ORDEN: DESCENDENTE**

Saguaro no Campo Norte da GOODS (NIRCam)

Com base em suas análises de estrelas espirais, cientistas propuseram um caminho que os pequenos pontos vermelhos podem seguir à medida que o universo amadurece...

29 DE JULHO DE 2020

[@ NASA](#)

Pequeno Ponto Vermelho no Desvio para o Vermelho 2: Gráfico Real e Simulado

Cientistas alteraram sinteticamente o desvio para o vermelho da galáxia espiral Saguaro, uma galáxia com desvio para o vermelho baixo, para um desvio para o vermelho mais alto, a fim de descobrir como ela apareceria...

29 DE JULHO DE 2020

[@ NASA](#)

Galáxia de Andrômeda

O Telescópio Espacial Hubble da NASA proporcionou uma visão detalhada de milhões de estrelas na galáxia de Andrômeda.

27 DE JULHO DE 2020

[@ NASA](#)

Hubble Espia Espiral Unilateral

Esta imagem do Hubble mostra a galáxia espiral NGC 4654, localizada a 72 milhões de anos-luz de distância na constelação de Virgem.

24 DE JULHO DE 2020

[@ NASA](#)

Imagem de contexto de Omega Centauri

Astrônomos descobriram o primeiro buraco negro de massa estelar em Omega Centauri, que possui uma estrela companheira visível, mostrada em detalhes.

13 DE JULHO DE 2020

[@ NASA](#)

O telescópio Hubble da NASA avista um projétil estelar durante o dia 4 de julho.

Estrelas vermelhas, brancas e azuis brilham como uma vela de aniversário sendo agitada em uma noite escura nesta nova imagem de...

4 DE JULHO DE 2020

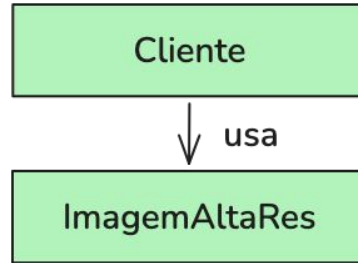
[@ NASA](#)

300 x 450 MB = 132 GB

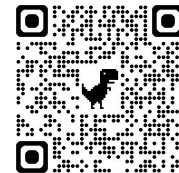


Proxy - O Problema

- O cliente conversa diretamente com a imagem.



- Criar o objeto já significa carregar o arquivo.



Proxy - O Problema

- O inicializador (`__init__`) acopla o processo de existir a estar carregado

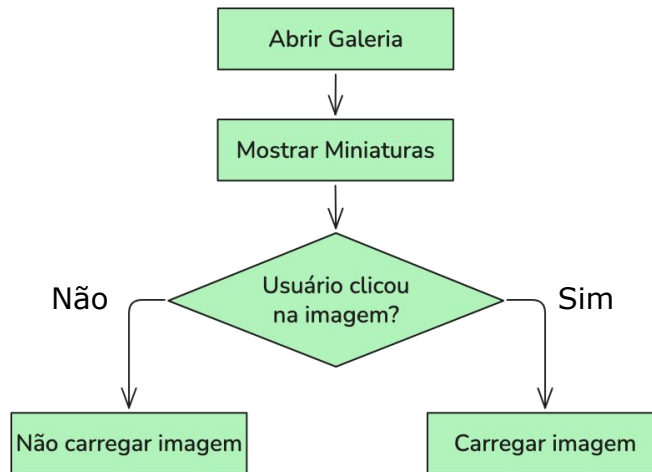
```
1 import time
2
3 class ImagemAltaResolucao:
4     def __init__(self, caminho: str):
5         self.caminho = caminho
6         self._pixels = self._carregar_do_disco() # ⚠️
7
8     def _carregar_do_disco(self) -> bytes:
9         print(f"[DISCO] Lendo {self.caminho} (450 MB)...")
10        time.sleep(2) # simula I/O
11        return b"..." * 1000
12
13    def exibir(self) -> None:
14        print(f"[TELA] Exibindo {self.caminho}")
15
16 # Abrindo a galeria:
17 caminhos = [f"hubble_{i:03}.fits" for i in range(300)]
18 galeria = [ImagemAltaResolucao(c) for c in caminhos] #600s e 132 GB
```

- Não há como criar a lista qualquer sem pagar o custo total de instanciação de objetos



Uma possível solução

- Uma ideia seria carregar somente quando necessário

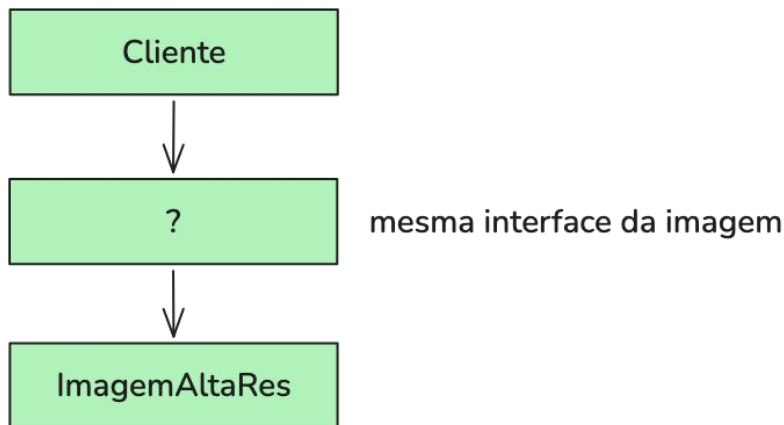


- Como implementar esse comportamento sem alterar toda a aplicação?



Proxy - O Problema

- Em vez de entregar diretamente a imagem de alta resolução ao cliente, podemos entregar um objeto com exatamente a mesma interface.



- O cliente perceberia a diferença?
 - Se a interface é idêntica então não.
- Existe um padrão criado exatamente para representar outro objeto e controlar o acesso a ele: **Proxy**.



Proxy

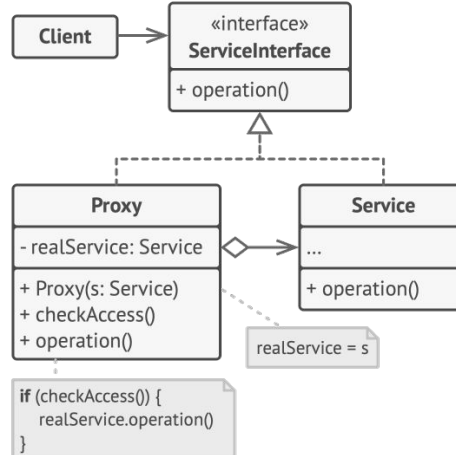
"Fornecer um objeto representante de outro objeto para controlar o acesso a esse objeto."

- Padrão estrutural
 - Também chamado de **Surrogate**.
- O proxy tem a mesma interface do objeto real
 - É intercambiável com ele
- O cliente não fala com o objeto real, fala com o proxy, sem saber disso
- O proxy pode:
 - adiar a criação do objeto original (lazy loading)
 - negar o acesso
 - registrar log, ou
 - falar com um objeto que está em outra máquina



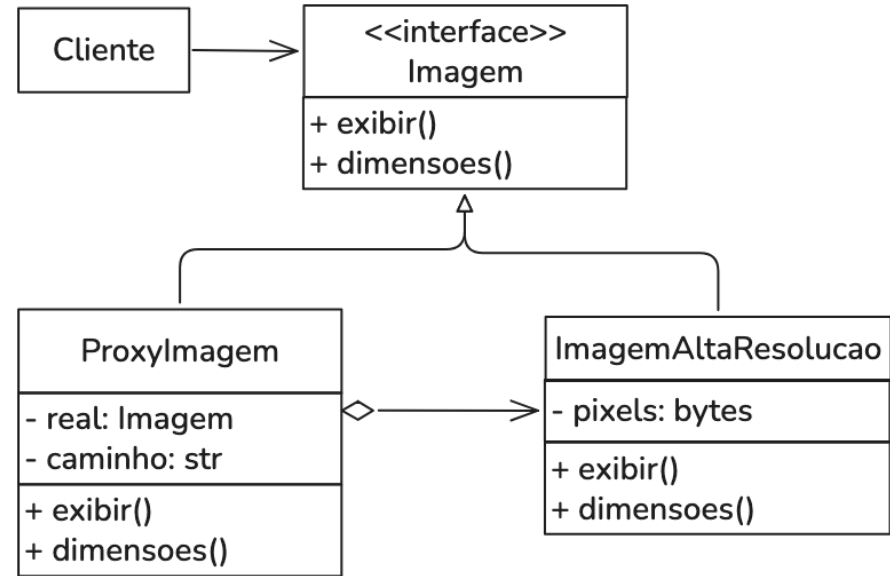
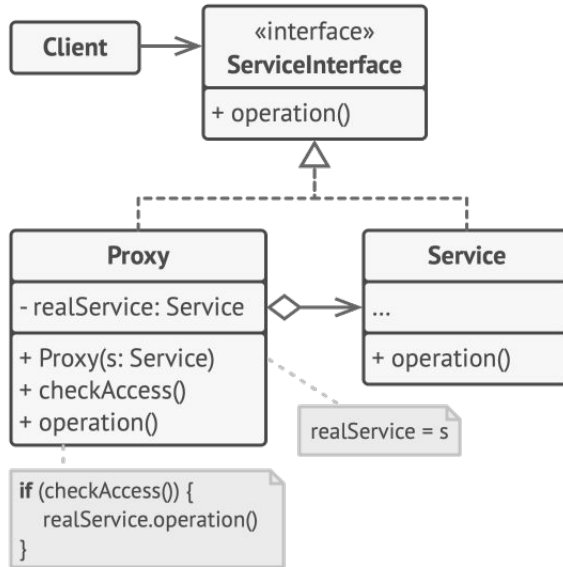
Estrutura do Proxy

1. **ServiceInterface** declara a interface do serviço.
 - a. O proxy deve segui-la para conseguir se disfarçar de objeto do serviço
2. **Service** é a classe que contém a lógica de negócio útil (e custosa)
3. **Proxy** guarda um campo de referência para um objeto do serviço e implementa a mesma interface
4. **Client** trabalha com serviços e proxies através da mesma interface.
 - a. é isso que torna a troca invisível.

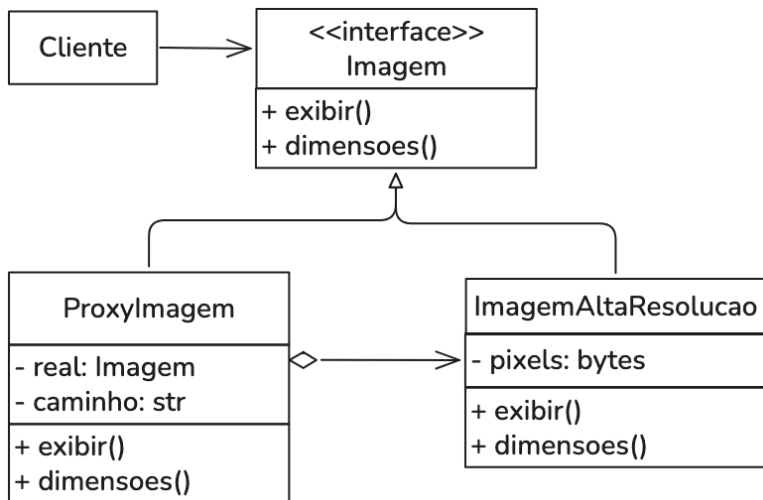




Aplicando ao nosso domínio

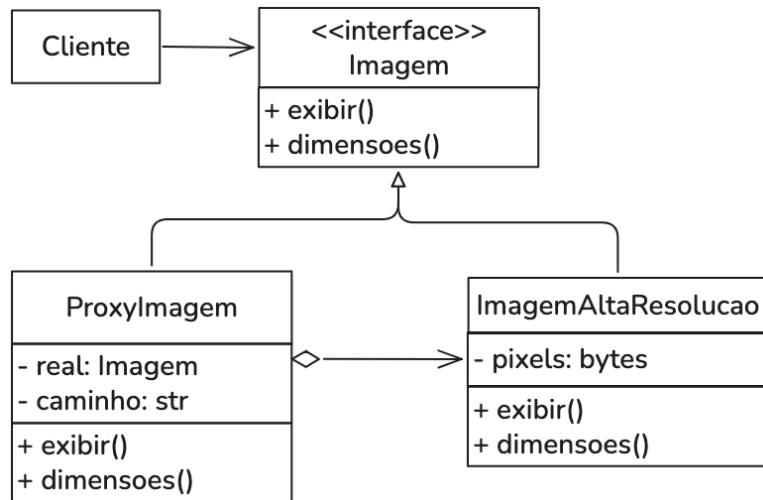
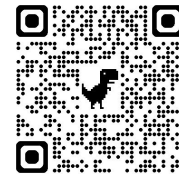


Aplicando ao nosso domínio



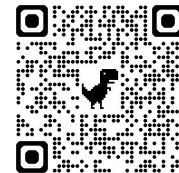
```
1 from abc import ABC, abstractmethod
2 import time
3
4 class Imagem(ABC): #ServiceInterface
5     @abstractmethod
6     def exibir(self) -> None: ...
7
8     @abstractmethod
9     def dimensoes(self) -> tuple[int, int]: ...
10
11
12 class ImagemAltaResolucao(Imagem): #Service
13     def __init__(self, caminho: str):
14         self.caminho = caminho
15         self._pixels = self._carregar_do_disco()
16
17     def _carregar_do_disco(self) -> bytes:
18         print(f"[DISCO] Lendo {self.caminho} (450 MB)...")
19         time.sleep(2)
20         return b"\x89PNG..." * 1000
21
22     def exibir(self) -> None:
23         print(f"[TELA] Exibindo {self.caminho}")
24
25     def dimensoes(self) -> tuple[int, int]:
26         return (8192, 8192)
```

Aplicando ao nosso domínio



```
1 class ProxyImagem(Imagem): #Proxy
2     def __init__(self, caminho: str, largura: int, altura: int):
3         self.caminho = caminho
4         self._dimensoes = (largura, altura)
5         self._real: ImagemAltaResolucao = None
6
7     def dimensoes(self) -> tuple[int, int]:
8         return self._dimensoes
9
10    def exibir(self) -> None:
11        if self._real is None:
12            print(f"[PROXY] 1ª exibição de {self.caminho} - instanciando o real")
13            self._real = ImagemAltaResolucao(self.caminho)
14            self._real.exibir()
```

Testando



```
1 def abrir_galeria(imagens: list[Imagem]) -> None:
2     for img in imagens:
3         print(f"miniatura: {img.caminho} {img.dimensoes()}")
4
5 galeria: list[Imagem] = [
6     ProxyImagem("hubble_001.fits", 8192, 8192),
7     ProxyImagem("hubble_002.fits", 8192, 8192),
8     ProxyImagem("hubble_003.fits", 8192, 8192),
9 ]
10
11 abrir_galeria(galeria)
12 galeria[1].exibir()
13 galeria[1].exibir()
```

```
1 miniatura: hubble_001.fits (8192, 8192)
2 miniatura: hubble_002.fits (8192, 8192)
3 miniatura: hubble_003.fits (8192, 8192)
4 [PROXY] 1ª exibição de hubble_002.fits → instanciando o real
5 [DISCO] Lendo hubble_002.fits (450 MB)...
6 [TELA]  Exibindo hubble_002.fits
7 [TELA]  Exibindo hubble_002.fits
```



Proxy para proteção do objeto

```
1 class AcessoNegado(Exception): ...
2
3 class ProxyProtecaoImagem(Imagem):
4     PAPEIS_AUTORIZADOS = {"pesquisador_senior", "admin"}
5
6     def __init__(self, real: Imagem, papel_usuario: str):
7         self._real = real
8         self._papel = papel_usuario
9
10    def dimensoes(self) -> tuple[int, int]:
11        return self._real.dimensoes()
12
13    def exibir(self) -> None:
14        if self._papel not in self.PAPEIS_AUTORIZADOS:
15            raise AcessoNegado(f"papel '{self._papel}' não pode abrir o RAW")
16        print(f"[AUDIT] acesso autorizado a {self._papel}")
17        self._real.exibir()
```

```
1 img = ProxyProtecaoImagem(ProxyImagem("hubble_007.fits", 8192, 8192), "visitante")
2 img.dimensoes() # (8192, 8192)
3 img.exibir() # AcessoNegado: papel 'visitante' não pode abrir o RAW
```



Prós e Contras do Proxy

Prós

- Controle sobre o objeto real sem que o cliente saiba
- O objeto real pode nem existir ainda (ou estar em outro servidor)
- Funciona mesmo quando o código do serviço não pode ser alterado (biblioteca de terceiros).
- Respeita o **O** (Open/Closed) e o **S** (Single Responsibility) do **SOLID**

Contras

- Mais uma classe e mais um nível de abstração
- A resposta ao cliente pode atrasar de forma imprevisível
- Se o cliente depende da classe concreta da pra descobrir o proxy

Não use:

- Se o custo do objeto é irrelevante -> + complexidade por nada
- Se o que você quer é acrescentar comportamento ao objeto original



O Problema - Novos requisitos

Até aqui tudo certo, mas... 2 semanas depois chegaram novos requisitos:

- Imagens públicas precisam sair com marca d'água do observatório
- Imagens exibidas no celular precisam de compressão
- Toda exibição precisa de log de auditoria
- Algumas imagens precisam de ajuste de contraste antes de aparecer

O cliente é quem escolhe, **em tempo de execução**, quais desses tratamentos se aplicam a cada imagem

- Marca d'água + log
- Compressão + contraste + marca d'água
- Nenhum
- Todos



O Problema - Novos requisitos

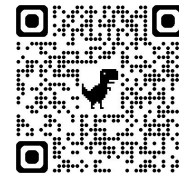
Vamos tentar resolver por herança

- `class ImagemComMarcaDagua(ImagemAltaResolucao): ...`
- `class ImagemComprimida(ImagemAltaResolucao): ...`
- `class ImagemComLog(ImagemAltaResolucao): ...`
- `class ImagemComMarcaDaguaEComprimida(ImagemAltaResolucao): ...`
- `class ImagemComMarcaDaguaELog(ImagemAltaResolucao): ...`
- `class ImagemComprimidaComLog(ImagemAltaResolucao): ...`
- `class ImagemComMarcaDaguaComprimidaComLog(ImagemAltaResolucao): ...`
- `class ImagemComContrasteEMarcaDagua(ImagemAltaResolucao): ...`

Para n comportamentos combináveis teremos $2^n - 1$ subclasses, temos 4 requisitos... logo...

Isso viola o princípio:

- "favoreça composição de objetos em vez de herança de classe"

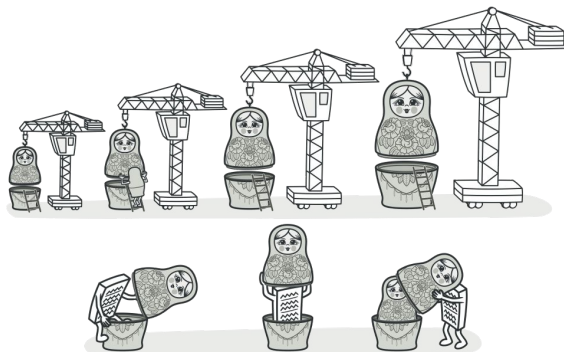


Decorator

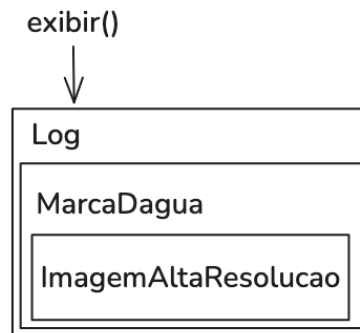
"Agregar dinamicamente responsabilidades adicionais a um objeto."

Fornecer uma alternativa flexível ao uso de subclasses para extensão de funcionalidade.."

- Padrão estrutural.
 - Também chamado de **Wrapper**.
- Em vez de criar uma subclasse para cada combinação, criar um objeto que contém outro objeto do mesmo tipo e acrescenta algo ao redor da chamada.
- Como o invólucro tem a mesma interface do que ele envolve, um invólucro pode envolver outro invólucro.



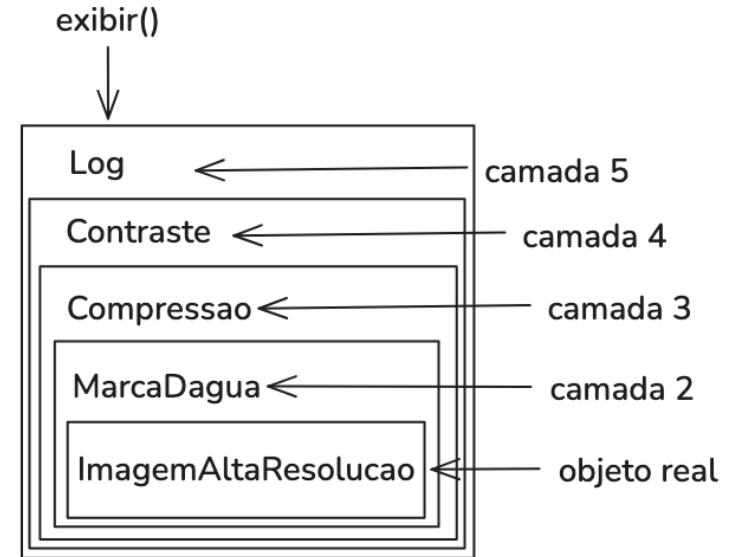
carlos.melo@upe.br





Decorator

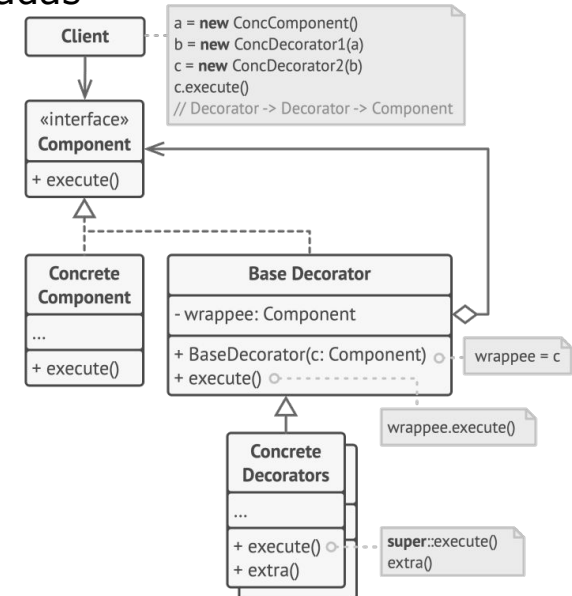
- 4 Comportamentos que virariam 15 subclasses viraram 4 decoradores e qualquer combinação é possível de ser montada





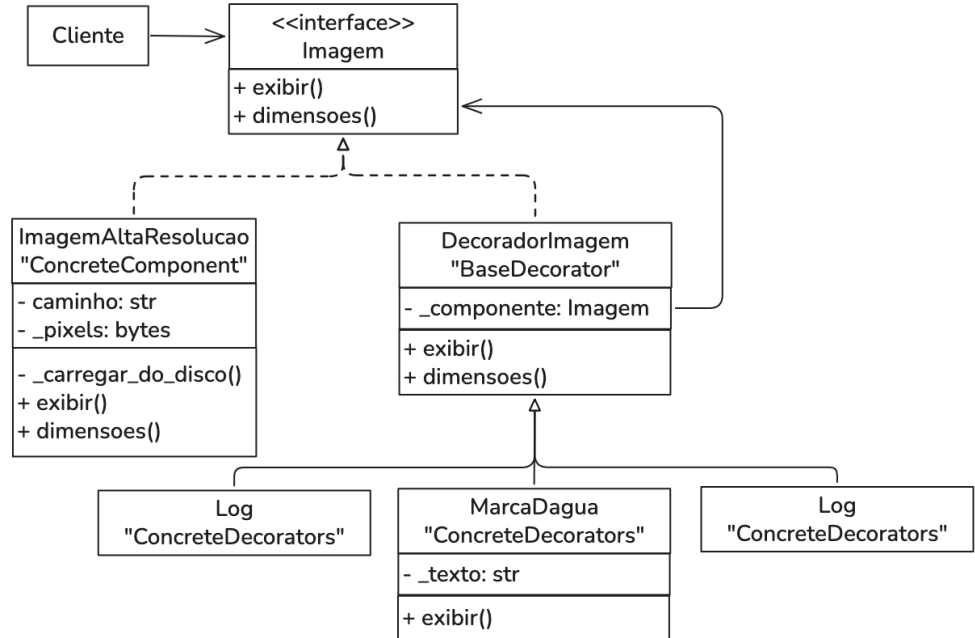
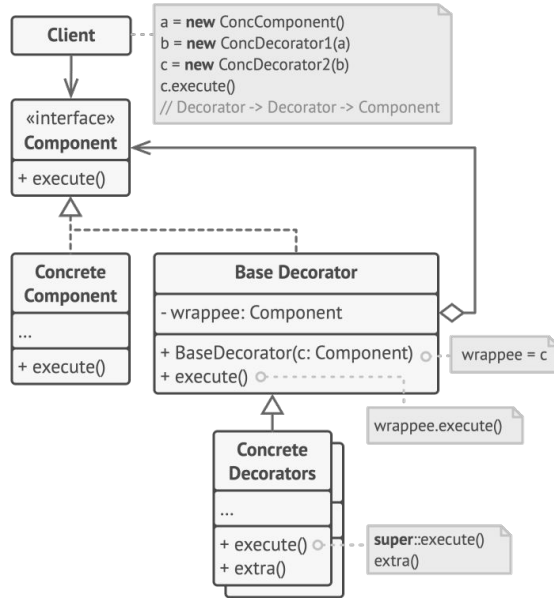
Estrutura do Decorator

1. O **Component** declara a interface comum tanto aos envoltórios quanto objetos
2. O **ConcreteComponent** é uma subclasse de objetos sendo envolvidos
3. **BaseDecorator** tem um campo para referenciar um objeto envolvido
4. Os **ConcreteDecorators** definem os comportamentos adicionais
5. O **Client** pode envolver componentes em múltiplas camadas





Aplicando ao nosso domínio



Implemente o Decorator no nosso domínio



Prós e Contras do Decorator

Prós

- Estende comportamento sem subclasses e sem tocar na classe original
- Combinações escolhidas em tempo de execução
- Camadas podem até ser removidas
- Evita explosão de subclasses
- Respeita o **S** (Single Responsibility) do **SOLID**

Contras

- Muitos objetos pequenos e parecidos tornam a pilha difícil de depurar
- Remover uma camada específica do meio da pilha é trabalhoso
- O comportamento final depende da ordem implicando em dependência implícita
- O cliente perde a identidade concreta uma vez que `isinstance(img, ImagemAltaResolucao)` é `False`.

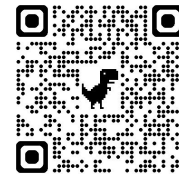
Não use:

- Se os comportamentos nunca se combinam ou nunca variam em *runtime*, herança simples é mais barata e legível.



Prós e Contras do Decorator

- Decorator != @decorator
- @decorator é para funções e o padrão é para instâncias de objetos
- @decorator funciona em tempo de importação e o padrão de projetos Decorator funciona em tempo de execução



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br